

udemy - Deployment

Monday, December 8, 2025 10:15 AM

Question 1

When running a Rolling deployment in Elastic Beanstalk environment, only two batches completed the deployment successfully, while rest of the batches failed to deploy the updated version. Following this, the development team terminated the instances from the failed deployment. What will be the status of these failed instances post termination?

- Elastic Beanstalk will replace the failed instances with instances running the application version from the most recent successful deployment
- Elastic Beanstalk will replace the failed instances after the application version to be installed is manually chosen from AWS Console
- Elastic Beanstalk will not replace the failed instances
- Elastic Beanstalk will replace the failed instances with instances running the application version from the oldest successful deployment

Overall explanation

Correct option:

Elastic Beanstalk will replace them with instances running the application version from the most recent successful deployment

When processing a batch, Elastic Beanstalk detaches all instances in the batch from the load balancer, deploys the new application version, and then reattaches the instances. If you enable connection draining, Elastic Beanstalk drains existing connections from the Amazon EC2 instances in each batch before beginning the deployment.

If a deployment fails after one or more batches completed successfully, the completed batches run the new version of your application while any pending batches continue to run the old version.

You can identify the version running on the instances in your environment on the health page in the console. This page displays the deployment ID of the most recent deployment that was executed on each instance in your environment. If you terminate instances from the failed deployment, Elastic Beanstalk replaces them with instances running the application version from the most recent successful deployment.

Incorrect options:

Elastic Beanstalk will not replace the failed instances

Elastic Beanstalk will replace the failed instances with instances running the application version from the oldest successful deployment

Elastic Beanstalk will replace the failed instances after the application version to be installed is manually chosen from AWS Console

These three options contradict the explanation provided above, so these options are incorrect.

Reference:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html>

Question 2

As a developer, you are working on creating an application using AWS Cloud Development Kit (CDK).

Which of the following represents the correct order of steps to be followed for creating an app using AWS CDK?

- Create the app from a template provided by AWS CloudFormation -> Add code to the app to create resources within stacks -> Synthesize one or more stacks in the app -> Deploy stack(s) to your AWS account -> Build the app
- Create the app from a template provided by AWS CDK -> Add code to the app to create resources within stacks -> Synthesize one or more stacks in the app -> Deploy stack(s) to your AWS account -> Build the app
- Create the app from a template provided by AWS CDK -> Add code to the app to create resources within stacks -> Build the app (optional) -> Synthesize one or more stacks in the app -> Deploy stack(s) to your AWS account
- Create the app from a template provided by AWS CloudFormation -> Add code to the app to create resources within stacks -> Build the app (optional) -> Synthesize one or more stacks in the app -> Deploy stack(s) to your AWS account

Overall explanation

Correct option:

Create the app from a template provided by AWS CDK -> Add code to the app to create resources within stacks -> Build the app (optional) -> Synthesize one or more stacks in the app -> Deploy stack(s) to your AWS account

The standard AWS CDK development workflow is similar to the workflow you're already familiar as a developer. There are a few extra steps:

1. Create the app from a template provided by AWS CDK - Each AWS CDK app should be in its own directory, with its own local module dependencies. Create a new directory for your app. Now initialize the app using the `cdk init` command, specifying the desired template ("app") and programming language. The `cdk init` command creates a number of files and folders inside the created home directory to help you organize the source code for your AWS CDK app.
2. Add code to the app to create resources within stacks - Add custom code as is needed for your application.
3. Build the app (optional) - In most programming environments, after making changes to your code, you'd build (compile) it. This isn't strictly necessary with the AWS CDK—the Toolkit does it for you so you can't forget. But you can still build manually whenever you want to catch syntax and type errors.
4. Synthesize one or more stacks in the app to create an AWS CloudFormation template - Synthesize one or more stacks in the app to create an AWS CloudFormation template. The synthesis step catches logical errors in defining your AWS resources. If your app contains more than one stack, you'd need to specify which stack(s) to synthesize.
5. Deploy one or more stacks to your AWS account - It is optional (though good practice) to synthesize before deploying. The AWS CDK synthesizes your stack before each deployment. If your code has security implications, you'll see a summary of these and need to confirm them before deployment proceeds. `cdk deploy` is used to deploy the stack using CloudFormation templates. This command displays progress information as your stack is deployed. When it's done, the command prompt reappears.

Incorrect options:

Create the app from a template provided by AWS CloudFormation -> Add code to the app to create resources within stacks -> Build the app (optional) -> Synthesize one or more stacks in the app -> Deploy stack(s) to your AWS account

Create the app from a template provided by AWS CloudFormation -> Add code to the app to create resources within stacks -> Synthesize one or more stacks in the app -> Deploy stack(s) to your AWS account -> Build the app

For both these options, you cannot use AWS CloudFormation to create the app. So these options are incorrect.

Create the app from a template provided by AWS CDK -> Add code to the app to create resources within stacks -> Synthesize one or more stacks in the app -> Deploy stack(s) to your AWS account -> Build the app - You cannot have the build step after deployment. So this option is incorrect.

Reference:

https://docs.aws.amazon.com/cdk/latest/guide/hello_world.html

Question 3

A Developer at a company is working on a CloudFormation template to set up resources. Resources will be defined using code and provisioned based on certain conditions defined in the Conditions section.

Which section of a CloudFormation template cannot be associated with Condition?

- Resources
- Parameters
- Outputs
- Conditions

Overall explanation

Correct option:

Parameters

Parameters enable you to input custom values to your CloudFormation template each time you create or update a stack. Please see this note to understand how to define a parameter in a template:

Defining a parameter in a template

The following example declares a parameter named `InstanceTypeParameter`. This parameter lets you specify the Amazon EC2 instance type for the stack to use when you create or update the stack.

Note that `InstanceTypeParameter` has a default value of `t2.micro`. This is the value that AWS CloudFormation uses to provision the stack unless another value is provided.

JSON

```
"Parameters": {
  "InstanceTypeParameter": {
    "Type": "String",
    "Default": "t2.micro",
    "AllowedValues": [ "t2.micro", "m1.small", "m1.large" ],
    "Description": "Enter t2.micro, m1.small, or m1.large. Default is t2.micro."
  }
}
```

YAML

```
Parameters:
  InstanceTypeParameter:
    Type: String
    Default: t2.micro
    AllowedValues:
      - t2.micro
      - m1.small
      - m1.large
    Description: Enter t2.micro, m1.small, or m1.large. Default is t2.micro.
```

via - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/parameters-section-structure.html>

The optional `Conditions` section contains statements that define the circumstances under which entities are created or configured. For example, you can create a condition and then associate it with a resource or output so that AWS CloudFormation only creates the resource or output if the condition is true.

You might use conditions when you want to reuse a template that can create resources in different contexts, such as a test environment versus a production environment. In your template, you can add an `EnvironmentType` input parameter, which accepts either `prod` or `test` as inputs. For the production environment, you might include Amazon EC2 instances with certain capabilities; however, for the test environment, you want to use reduced capabilities to save money.

Conditions cannot be used within the `Parameters` section. After you define all your conditions, you can associate them with resources and resource properties only in the `Resources` and `Outputs` sections of a template.

Please review this note for more details:

Conditions

PDF Kindle RSS Filter View All

The optional `Conditions` section contains statements that define the circumstances under which entities are created or configured. For example, you can create a condition and then associate it with a resource or output so that AWS CloudFormation only creates the resource or output if the condition is true. Similarly, you can associate the condition with a property so that AWS CloudFormation only sets the property to a specific value if the condition is true. If the condition is false, AWS CloudFormation sets the property to a different value that you specify.

You might use conditions when you want to reuse a template that can create resources in different contexts, such as a test environment versus a production environment. In your template, you can add an `EnvironmentType` input parameter, which accepts either `prod` or `test` as inputs. For the production environment, you might include Amazon EC2 instances with certain capabilities; however, for the test environment, you want to use reduced capabilities to save money. With conditions, you can define which resources are created and how they're configured for each environment type.

Conditions are evaluated based on predefined pseudo parameters or input parameter values that you specify when you create or update a stack. Within each condition, you can reference another condition, a parameter value, or a mapping. After you define all your conditions, you can associate them with resources and resource properties in the `Resources` and `Outputs` sections of a template.

At stack creation or stack update, AWS CloudFormation evaluates all the conditions in your template before creating any resources. Resources that are associated with a true condition are created. Resources that are associated with a false condition are ignored. AWS CloudFormation also re-evaluates these conditions at each stack update before updating any resources. Resources that are still associated with a true condition are updated. Resources that are now associated with a false condition are deleted.

Important

During a stack update, you cannot update conditions by themselves. You can update conditions only when you include changes that add, modify, or delete resources.

via - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/conditions-section-structure.html>

Please visit <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/parameters-section-structure.html> for more information on the parameter structure.

Incorrect options:

Resources - `Resources` section describes the resources that you want to provision in your AWS CloudFormation stacks. You can associate conditions with the resources that you want to conditionally create.

Conditions - You actually define conditions in this section of the CloudFormation template

Outputs - The optional `Outputs` section declares output values that you can import into other stacks (to create cross-stack references), return in response (to describe stack calls), or view on the AWS CloudFormation console. For example, you can output the S3 bucket name for a stack to make the bucket easier to find. You can associate conditions with the outputs that you want to conditionally create.

References:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/parameters-section-structure.html>

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/conditions-section-structure.html>

Question 4

A global e-commerce company wants to perform geographic load testing of its order processing API. The company must deploy resources to multiple AWS Regions to support the load testing of the API.

How can the company address these requirements without additional application code?

- Set up an AWS CloudFormation template that defines the load test resources. Develop region-specific Lambda functions to create a stack from the AWS CloudFormation template in each Region when the respective function is invoked
- Set up an AWS CloudFormation template that defines the load test resources. Leverage the AWS CLI `create-stack-set` command to create a stack set in the desired Regions
- Set up an AWS Cloud Development Kit (CDK) Toolkit that defines the load test resources. Leverage the CDK CLI to create a stack from the template in each Region
- Set up an AWS Organizations template that defines the load test resources across the organization. Leverage the AWS CLI `create-stack-set` command to create a stack set in the desired Regions

Overall explanation

Correct option:

Set up an AWS CloudFormation template that defines the load test resources. Leverage the AWS CLI `create-stack-set` command to create a stack set in the desired Regions

AWS CloudFormation StackSets extends the capability of stacks by enabling you to create, update, or delete stacks across multiple accounts and AWS Regions with a single operation. Using an administrator account, you define and manage an AWS CloudFormation template, and use the template as the basis for provisioning stacks into selected target accounts across specified AWS Regions.



via - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/what-is-cfnstacksets.html>

A stack set lets you create stacks in AWS accounts across regions by using a single CloudFormation template. A stack set's CloudFormation template defines all the resources in each stack. As you create the stack set, specify the template to use, in addition to any parameters and capabilities that the template requires.

After you've defined a stack set, you can create, update, or delete stacks in the target accounts and AWS Regions you specify.

Stack instances

A stack instance is a reference to a stack in a target account within a Region. A stack instance can exist without a stack. For example, if the stack couldn't be created for some reason, the stack instance shows the reason for stack creation failure. A stack instance associated with only one stack set.

The following figure shows the logical relationship between stack sets, stack operations, and stacks. When you update a stack set, all associated stack instances update throughout all accounts and Regions.



Stack set operations

You can perform the following operations on stack sets.

Create stack set

Creating a new stack set includes specifying a CloudFormation template that you want to use to create stacks, specifying the target accounts to which you want to create stacks, and identifying the AWS Regions in which you want to deploy stacks in your target accounts. A stack set ensures consistent deployment of the same stack resources, with the same settings, in all specified target accounts within the Regions you choose.

Update stack set

When you update a stack set, you push changes out to stacks in your stack set. You can update a stack set in one of the following ways. Your template updates always affect all stacks you can't selectively update the template for some stacks in the stack set, but not others.

- Change existing settings in the template or add new resources, such as updating parameter settings for a specific service, or adding new Amazon EC2 instances.
- Replace the template with a different template.
- Add stacks in existing or additional target accounts, across existing or additional Regions.

Delete stacks

When you delete stacks, you are removing a stack and all its associated resources from the target accounts you specify, within the Regions you specify. You can delete stacks in the following ways.

- Delete stacks from some target accounts, while leaving other stacks in other target accounts running.
- Delete stacks from some Regions, while leaving stacks in other Regions running.
- Delete stacks from your stack set, but save them so they continue to run independently of your stack set by choosing the **Retain Stacks** option. You can then manage retained stacks outside of your stack set in AWS CloudFormation.
- Delete all stacks in your stack set, in preparation for deleting your entire stack set.

Delete stack set

You can delete your stack set only when there are no stack instances in it.

via - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stacksets-concepts.html>

Incorrect options:

Set up an AWS CloudFormation template that defines the load test resources. Develop region-specific Lambda functions to create a stack from the AWS CloudFormation template in each Region when the respective function is invoked - If you do not use a stack set, then you need to define the CloudFormation templates in each region as well as develop lambda functions in each region to create a stack from the corresponding CloudFormation template. This is unnecessary bloat that can be avoided by simply using the CloudFormation StackSets.

Set up an AWS Cloud Development Kit (CDK) Toolkit that defines the load test resources. Leverage the CDK CLI to create a stack from the template in each Region - AWS CDK is a framework for defining cloud infrastructure in code and provisioning it through AWS CloudFormation. The CDK Toolkit again poses regional limitations and is not the right fit for the given use case.

Set up an AWS Organizations template that defines the load test resources across the organization. Leverage the AWS CLI create-stack-set command to create a stack set in the desired Regions - This option acts as a distractor. AWS Organizations cannot be used to create templates for provisioning AWS infrastructure. AWS Organizations is an account management service that enables you to consolidate multiple AWS accounts into an organization that you create and centrally manage.

References:

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_introduction.html

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stacksets-concepts.html>

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/what-is-cfnstacksets.html>

Question 5

After a test deployment in ElasticBeanstalk environment, a developer noticed that all accumulated Amazon EC2 burst balances were lost. Which of the following options can lead to this behavior?

- When a canary deployment fails, it resets the EC2 burst balances to zero
- The deployment was run as a Rolling deployment, resulting in the resetting of EC2 burst balances
- The deployment was either run with immutable updates or in traffic splitting mode
- The deployment was run as a All-at-once deployment, flushing all the accumulated EC2 burst balances

Overall explanation

Correct option:

The deployment was either run with immutable updates or in traffic splitting mode - Immutable deployments perform an immutable update to launch a full set of new instances running the new version of the application in a separate Auto Scaling group, alongside the instances running the old version. Immutable deployments can prevent issues caused by partially completed rolling deployments.

Traffic-splitting deployments let you perform canary testing as part of your application deployment. In a traffic-splitting deployment, Elastic Beanstalk launches a full set of new instances just like during an immutable deployment. It then forwards a specified percentage of incoming client traffic to the new application version for a specified evaluation period.

If the deployment creates NEW EC2 instances → burst balance resets

If deployment only updates EXISTING instances → burst balance stays

Some policies replace all instances during the deployment or update. This causes all accumulated Amazon EC2 burst balances to be lost. It happens in the following cases:

- Managed platform updates with instance replacement enabled
- Immutable updates
- Deployments with immutable updates or traffic splitting enabled

Incorrect options:

The deployment was run as a Rolling deployment, resulting in the resetting of EC2 burst balances - With rolling deployments, Elastic Beanstalk splits the environment's Amazon EC2 instances into batches and deploys the new version of the application to one batch at a time. Rolling deployments do not result in loss of EC2 burst balances.

The deployment was run as a All-at-once deployment, flushing all the accumulated EC2 burst balances - The traditional All-at-once deployment, wherein all the instances are updated simultaneously, does not result in loss of EC2 burst balances.

When a canary deployment fails, it resets the EC2 burst balances to zero - This is incorrect and given only as a distractor.

Reference:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html>

Question 6

The Technical Lead of your team has reviewed a CloudFormation YAML template written by a new recruit and specified that an invalid section has been added to the template. Which of the following represents an invalid section of the CloudFormation template?

- 'Conditions' section of the template
- 'Dependencies' section of the template
- 'Resources' section of the template
- 'Parameters' section of the template

Overall explanation

Correct option:

Templates include update several major sections. The Resources section is the only required section.

Sample CloudFormation YAML template:

YAML

The following example shows a YAML-formatted template fragment.

```
AWSTemplateFormatVersion: "version_date"

Description:
  String

Metadata:
  template metadata

Parameters:
  set of parameters

Mappings:
  set of mappings

Conditions:
  set of conditions

Transforms:
  set of transforms

Resources:
  set of resources

Outputs:
  set of outputs
```

via - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-anatomy.html>

'Dependencies' section of the template - As you can see, there is no section called 'Dependencies' in the template. Although dependencies can be mentioned, there is no section itself for dependencies.

Incorrect options:

'Conditions' section of the template - This optional section includes conditions that control whether certain resources are created or whether certain resource properties are assigned a value during stack creation or update. For example, you could conditionally create a resource that depends on whether the stack is for a production or test environment.

'Resources' section of the template - This is the only required section and specifies the stack resources and their properties, such as an Amazon Elastic Compute Cloud instance or an Amazon Simple Storage Service bucket. You can refer to resources in the Resources and Outputs sections of the template.

'Parameters' section of the template - This optional section is helpful in passing Values to your template at runtime (when you create or update a stack). You can refer to parameters from the Resources and Outputs sections of the template.

Reference:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-anatomy.html>

Question 7

You have created a Java application that uses RDS for its main data storage and ElastiCache for user session storage. The application needs to be deployed using Elastic Beanstalk and every new deployment should allow the application servers to reuse the RDS database. On the other hand, user session data stored in ElastiCache can be re-created for every deployment. Which of the following configurations will allow you to achieve this? (Select two)

- RDS database defined in .ebextensions/
- RDS database defined externally and referenced through environment variables
- ElastiCache defined in .ebextensions/
- ElastiCache database defined externally and referenced through environment variables
- ElastiCache bundled with the application source code

Overall explanation

Correct option:

ElastiCache defined in .ebextensions/ - Any resources created as part of your .ebextensions is part of your Elastic Beanstalk template and will get deleted if the environment is terminated.

Advanced environment customization with configuration files (.ebextensions)

[PDF](#) | [Kindle](#)

You can add **AWS Elastic Beanstalk configuration files (.ebextensions)** to your web application's source code to **configure your environment and customize the AWS resources that it uses**. Configuration files are YAML- or JSON-formatted documents with a `.conf` file extension that you place in a folder named `.ebextensions` and deploy in your application source bundle.

Example

`ebextensions/network-load-balancer.conf`

This example makes a simple configuration change. It modifies a configuration option to set the type of your environment's load balancer to Network Load Balancer.

```
option: aws:elb:LoadBalancerType
value: NetworkLoadBalancer
```

We recommend using YAML for your configuration files, because it's more readable than JSON. YAML supports comments, multi-line comments, several alternatives for using quotes, and more. However, you can make any configuration change to Elastic Beanstalk configuration files directly using either JSON or JSON.

Note
When you are developing or testing new configuration files, launch a clean environment running the default application and deploy to that. Properly formatted configuration files will cause a new environment launch to fail unacceptably.

The `option: aws:elb:LoadBalancerType` section of a configuration file defines values for `configuration:options`. Configuration options let you configure your Elastic Beanstalk environment, the AWS resources it uses, and the software that runs your application. Configuration files are only one of several ways to set configuration options.

The `resources:options` section lets you further customize the resources in your application's environment, and define additional AWS resources beyond the functionality provided by configuration options. You can add and configure any resources supported by AWS CloudFormation, which Elastic Beanstalk uses to create environments.

The other sections of a configuration file (packages, source, files, users, groups, commands, container_commands, and service_commands) let you configure the EC2 instances that are launched in your environment. Whenever a server is launched in your environment, Elastic Beanstalk runs the operations defined in these sections to prepare the operating system and storage system for your application.

via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/ebextensions.html>

RDS database defined externally and referenced through environment variables - To decouple your database instance from your environment, you can run a database instance in Amazon RDS and configure your application to connect to it on launch. This enables you to connect multiple environments to a database, terminate an environment without affecting the database, and perform seamless updates with blue-green deployments. To allow the Amazon EC2 instances in your environment to connect to an outside database, you can configure the environment's Auto Scaling group with an additional security group.

Using Elastic Beanstalk with Amazon RDS:

Using Elastic Beanstalk with Amazon RDS

[PDF](#) | [Kindle](#)

AWS Elastic Beanstalk provides support for running **Amazon Relational Database Service (Amazon RDS)** instances in your Elastic Beanstalk environment. To learn about that, see [Adding a database to your Elastic Beanstalk environment](#). This works great for development and testing environments. However, it isn't ideal for a production environment because it ties the lifecycle of the database instance to the lifecycle of your application's environment.

Note
If you haven't used a DB instance with your application before, try adding one to a test environment with the Elastic Beanstalk console first. This lets you verify that your application is able to read environment properties, construct a connection string, and connect to a DB instance before you add Amazon Virtual Private Cloud (Amazon VPC) and security group configuration to the mix. See [Adding a database to your Elastic Beanstalk environment](#) for details.

To decouple your database instance from your environment, you can run a database instance in Amazon RDS and configure your application to connect to it on launch. This enables you to connect multiple environments to a database, terminate an environment without affecting the database, and perform seamless updates with blue-green deployments. For a detailed procedure, see [How do I decouple an Amazon RDS instance from an Elastic Beanstalk environment without downtime, database sync issues, or data loss?](#)

To allow the Amazon EC2 instances in your environment to connect to an outside database, you can configure the environment's Auto Scaling group with an additional security group. The security group that you attach to your environment can be the same one that is attached to your database instance, or a separate security group from which the database's security group allows ingress.

Note
You can connect your environment to a database by adding a rule to your database's security group that allows ingress from the autogenerated security group that Elastic Beanstalk attaches to your environment's Auto Scaling group. However, doing so creates a dependency between the two security groups. Subsequently, when you attempt to terminate the environment, Elastic Beanstalk will be unable to delete the environment's security group because the database's security group is dependent on it.

via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/AWSHowTo.RDS.html>

Incorrect options:

ElastiCache bundled with the application source code - ElastiCache is an AWS service and cannot be bundled with the source code.

RDS database defined in .ebextensions/ - The lifetime of the RDS instance gets tied to the lifetime of the Elastic Beanstalk environment, so this option is incorrect.

ElastiCache database defined externally and referenced through environment variables - For the given use-case, the client is fine with losing user session data and hence defining it in .ebextensions/ is more appropriate.

Question 8

You are a developer in a manufacturing company that has several servers on-site. The company decides to move new development to the cloud using serverless technology. You decide to use the AWS Serverless Application Model (AWS SAM) and work with an AWS SAM template file to represent your serverless architecture.

Which of the following is NOT a valid serverless resource type?

- AWS::Serverless::UserPool
- AWS::Serverless::SimpleTable
- AWS::Serverless::Function
- AWS::Serverless::Api

Overall explanation

Correct option:

AWS::Serverless::UserPool

The AWS Serverless Application Model (SAM) is an open-source framework for building serverless applications. It provides shorthand syntax to express functions, APIs, databases, and event source mappings. With just a few lines per resource, you can define the application you want and model it using YAML.

SAM supports the following resource types:

AWS::Serverless::Api
AWS::Serverless::Application
AWS::Serverless::Function
AWS::Serverless::HttpApi
AWS::Serverless::LayerVersion
AWS::Serverless::SimpleTable
AWS::Serverless::StateMachine

UserPool applies to the Cognito service which is used for authentication for mobile app and web. There is no resource named UserPool in the Serverless Application Model.

Incorrect options:

AWS::Serverless::Function - This resource creates a Lambda function, IAM execution role, and event source mappings that trigger the function.

AWS::Serverless::Api - This creates a collection of Amazon API Gateway resources and methods that can be invoked through HTTPS endpoints. It is useful for advanced use cases where you want full control and flexibility when you configure your APIs.

AWS::Serverless::SimpleTable - This creates a DynamoDB table with a single attribute primary key. It is useful when data only needs to be accessed via a primary key.

Reference:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/sam-specification-resources-and-properties.html>

Question 9

A developer wants to package the code and dependencies for the application-specific Lambda functions as container images to be hosted on Amazon Elastic Container Registry (ECR). Which of the following options are correct for the given requirement? (Select two)

- You can test the containers locally using the Lambda Runtime API
- You can deploy Lambda function as a container image, with a maximum size of 15 GB
- To deploy a container image to Lambda, the container image must implement the Lambda Runtime API
- Lambda supports both Windows and Linux-based container images
- AWS Lambda service does not support Lambda functions that use multi-architecture container images

Overall explanation

Correct options:

To deploy a container image to Lambda, the container image must implement the Lambda Runtime API - To deploy a container image to Lambda, the container image must implement the Lambda Runtime API. The AWS open-source runtime interface clients implement the API. You can add a runtime interface client to your preferred base image to make it compatible with Lambda.

AWS Lambda service does not support Lambda functions that use multi-architecture container images - Lambda provides multi-architecture base images. However, the image you build for your function must target only one of the architectures. Lambda does not support functions that use multi-architecture container images.

Incorrect options:

Lambda supports both Windows and Linux-based container images - Lambda currently supports only Linux-based container images.

You can test the containers locally using the Lambda Runtime API - You can test the containers locally using the Lambda Runtime Interface Emulator.

You can deploy Lambda function as a container image, with a maximum size of 15 GB - You can deploy Lambda function as container image with the maximum size of 10GB.

Reference:

<https://docs.aws.amazon.com/lambda/latest/dg/images-create.html>

Question 10

The development team at an e-commerce company completed the last deployment for their application at a reduced capacity because of the deployment policy. The application took a performance hit because of the traffic spike due to an on-going sale.

Which of the following represents the BEST deployment option for the upcoming application version such that it maintains at least the FULL capacity of the application and MINIMAL impact of failed deployment?

- Deploy the new application version using 'Rolling' deployment policy
- Deploy the new application version using 'Immutable' deployment policy
- Deploy the new application version using 'All at once' deployment policy
- Deploy the new application version using 'Rolling with additional batch' deployment policy

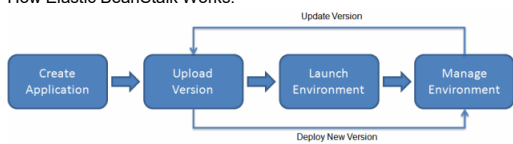
Overall explanation

Correct option:

Deploy the new application version using 'Immutable' deployment policy

With Elastic Beanstalk, you can quickly deploy and manage applications in the AWS Cloud without having to learn about the infrastructure that runs those applications. Elastic Beanstalk reduces management complexity without restricting choice or control. You simply upload your application, and Elastic Beanstalk automatically handles the details of capacity provisioning, load balancing, scaling, and application health monitoring.

How Elastic BeanStalk Works:



via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/Welcome.html>

The 'Immutable' deployment policy ensures that your new application version is always deployed to new instances, instead of updating existing instances. It also has the additional advantage of a quick and safe rollback in case the deployment fails. In an immutable update, a second Auto Scaling group is launched in your environment and the new version serves traffic alongside the old version until the new instances pass health checks. In case of deployment failure, the new instances are terminated, so the impact is minimal.

Overview of Elastic Beanstalk Deployment Policies:

- The following list provides summary information about the different deployment policies and adds related considerations.
- **All at once** - **The quickest deployment method.** Suitable if you can accept a short loss of service, and if quick deployments are important to you. With this method, Elastic Beanstalk deploys the new application version to each instance. Then, the web proxy or application server might need to restart. As a result, your application might be unavailable to users for a few seconds.
 - **Rolling** - **Avoids downtime and maintains reduced availability, at a cost of a longer deployment time.** Suitable if you can't accept any period of completely lost service. With this method, your application is deployed to your environment one batch of instances at a time. Most bandwidth is retained throughout the deployment.
 - **Rolling with additional batch** - **Avoids any reduced availability, at a cost of an even longer deployment time compared to the Rolling method.** Suitable if you must maintain the same bandwidth throughout the deployment. With this method, Elastic Beanstalk launches an extra batch of instances, then performs a rolling deployment. Launching the extra batch takes time, and ensures that the same bandwidth is retained throughout the deployment.
 - **Immutable** - **A quick deployment method that ensures your new application version is always deployed to new instances, instead of updating existing instances.** It also has the additional advantage of a quick and safe rollback in case the deployment fails. With this method, Elastic Beanstalk performs an *immutable update* to deploy your application. In an immutable update, a second Auto Scaling group is launched in your environment and the new version serves traffic alongside the old version until the new instances pass health checks.
 - **Traffic splitting** - **A canary testing deployment method.** Suitable if you want to test the health of your new application version using a portion of incoming traffic, while keeping the rest of the traffic served by the old application version.

The following table compares deployment method properties.

Deployment methods					
Method	Impact of failed deployment	Deploy time	Zero downtime	No AWS change	Rollback process
All at once	Downtime	⏱	❌ No	✅ Yes	Manual redeploy
Rolling	Single batch out of service; any successful batches before failure serving new application version	⏱	✅ Yes	✅ Yes	Manual redeploy
Rolling with an additional batch	Minimal if first batch fails; otherwise, similar to Rolling	⏱	✅ Yes	✅ Yes	Manual redeploy
Immutable	Minimal	⏱	✅ Yes	✅ Yes	Terminate new instances
Traffic splitting	Percentage of client traffic routed to new version temporarily impacted	⏱	✅ Yes	✅ Yes	Re-route traffic and terminate new instances
Blue/green	Minimal	⏱	✅ Yes	❌ No	Swap URLs

via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.deploy-existing-version.html>

Incorrect options:

Deploy the new application version using 'All at once' deployment policy - Although 'All at once' is the quickest deployment method, but the application may become unavailable to users (or have low availability) for a short time. Also in case of deployment failure, the application sees a downtime, so this option is not correct.

Deploy the new application version using 'Rolling' deployment policy - This policy avoids downtime and minimizes reduced availability, at a cost of a longer deployment time. However in case of deployment failure, the rollback process is via manual redeploy, so it's not as quick as the Immutable deployment.

Deploy the new application version using 'Rolling with additional batch' deployment policy - This policy avoids any reduced availability, at a cost of an even longer deployment time compared to the Rolling method. Suitable if you must maintain the same bandwidth throughout the deployment. However in case of deployment failure, the rollback process is via manual redeploy, so it's not as quick as the Immutable deployment.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/Welcome.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.deploy-existing-version.html>

Question 11

You are a developer working on a web application written in Java and would like to use AWS Elastic Beanstalk for deployment because it would handle deployment, capacity provisioning, load balancing, auto-scaling, and application health monitoring. In the past, you connected to your provisioned instances through SSH to issue configuration commands. Now, you would like a configuration mechanism that automatically applies settings for you.

Which of the following options would help do this?

- Use an AWS Lambda hook
- Use SSM parameter store as an input to your Elastic Beanstalk Configurations
- Deploy a CloudFormation wrapper

- Include config files in .ebextensions/ at the root of your source code

Overall explanation

Correct option:

Include config files in .ebextensions/ at the root of your source code

The option_settings section of a configuration file defines values for configuration options. Configuration options let you configure your Elastic Beanstalk environment, the AWS resources in it, and the software that runs your application. Configuration files are only one of several ways to set configuration options.

Advanced environment customization with configuration files
(.ebextensions)

PDF | Kindle

You can add AWS Elastic Beanstalk configuration files (.ebextensions) to your own application's source code to configure your environment and customize the AWS resources that it contains. Configuration files are YAML- or JSON-formatted documents with a .config file extension that you place in a folder named .ebextensions and deploy in your application source bundle.

Example
ebextensions/network-load-balancer.config

This example makes a simple configuration change. It modifies a configuration option to set the type of your environment's load balancer to Network Load Balancer.

```
option_settings:
  aws:elasticbeanstalk:environment:
    LoadBalancerType: network
```

We recommend using YAML for your configuration files, because it's more readable than JSON. YAML supports comments, multi-line commands, several alternatives for using quotes, and more. However, you can make any configuration change in Elastic Beanstalk configuration files identically using either YAML or JSON.

Tip
When you are developing or testing new configuration files, launch a clean environment running the default application and deploy to that. Poorly formatted configuration files will cause a new environment launch to fail unacceptably.

via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/ebextensions.html>

Incorrect options:

Deploy a CloudFormation wrapper - This is a made-up option. This has been added as a distractor.

Use SSM parameter store as an input to your Elastic Beanstalk Configurations - SSM parameter is still not supported for Elastic Beanstalk. So this option is incorrect.

Use an AWS Lambda hook - Lambda functions are not the best-fit to trigger these configuration changes as it would involve significant development effort.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/ebextensions.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/ebextensions-optionsettings.html>

Question 12

An e-commerce company manages a microservices application that receives orders from various partners through a customized API for each partner exposed via Amazon API Gateway. The orders are processed by a shared Lambda function.

How can the company notify each partner regarding the status of their respective orders in the most efficient manner, without affecting other partners' orders? Also, the solution should be scalable to accommodate new partners with minimal code changes required.

- Set up a separate SNS topic for each partner. Modify the Lambda function to publish messages for each partner to the partner's SNS topic
- Set up a separate Lambda function for each partner. Set up an SNS topic and subscribe each partner to the SNS topic. Modify each partner's Lambda function to publish messages with specific attributes to the SNS topic and apply the appropriate filter policy to the topic subscriptions
- Set up an SNS topic and subscribe each partner to the SNS topic. Modify the Lambda function to publish messages with specific attributes to the SNS topic and apply the appropriate filter policy to the topic subscriptions
- Set up a separate SNS topic for each partner and subscribe each partner to the respective SNS topic. Modify the Lambda function to publish messages with specific attributes to the partner's SNS topic and apply the appropriate filter policy to the topic subscriptions

Overall explanation

Correct option:

Set up an SNS topic and subscribe each partner to the SNS topic. Modify the Lambda function to publish messages with specific attributes to the SNS topic and apply the appropriate filter policy to the topic subscriptions

An Amazon SNS topic is a logical access point that acts as a communication channel. A topic lets you group multiple endpoints (such as AWS Lambda, Amazon SQS, HTTP/S, or an email address). For example, to broadcast the messages of a message-producer system (such as, an e-commerce website) working with multiple other services that require its messages (for example, checkout and fulfillment systems), you can create a topic for your producer system.

By default, an Amazon SNS topic subscriber receives every message that's published to the topic. To receive only a subset of the messages, a subscriber must assign a filter policy to the topic subscription. A filter policy is a JSON object containing properties that define which messages the subscriber receives. Amazon SNS supports policies that act on the message attributes or the message body, according to the filter policy scope that you set for the subscription. Filter policies for the message body assume that the message payload is a well-formed JSON object.

For the given use case, you can change the Lambda function to publish messages with specific attributes to the single SNS topic and apply the appropriate filter policy to the topic subscriptions for each of the partners. This is also easily scalable for new partners since only the filter policy needs to be set up for the new partner.

Incorrect options:

Set up a separate SNS topic for each partner. Modify the Lambda function to publish messages for each partner to the partner's SNS topic

Set up a separate SNS topic for each partner and subscribe each partner to the respective SNS topic. Modify the Lambda function to publish messages with specific attributes to the partner's SNS topic and apply the appropriate filter policy to the topic subscriptions

Both of these options represent an inefficient solution as there is no need to segregate each partner's updates into a separate SNS topic. A single SNS topic with distinct filter policies is sufficient.

Set up a separate Lambda function for each partner. Set up an SNS topic and subscribe each partner to the SNS topic. Modify each partner's Lambda function to publish messages with specific attributes to the SNS topic and apply the appropriate filter policy to the topic subscriptions - This is again an inefficient solution as there is no need to create a separate Lambda function for each partner as just a shared Lambda function is sufficient to process the orders and send an update to the single SNS topic with distinct filter policies.

References:

<https://docs.aws.amazon.com/sns/latest/dg/sns-create-topic.html>

<https://docs.aws.amazon.com/sns/latest/dg/sns-message-filtering.html>

Question 13

Your company has embraced cloud-native microservices architectures. New applications must be dockerized and stored in a registry service offered by AWS. The architecture should support dynamic port mapping and support multiple tasks from a single service on the same container instance. All services should run on the same EC2 instance.

Which of the following options offers the best-fit solution for the given use-case?

- Classic Load Balancer + ECS
- Application Load Balancer + Beanstalk
- Application Load Balancer + ECS
- Classic Load Balancer + Beanstalk

Overall explanation

Correct option:

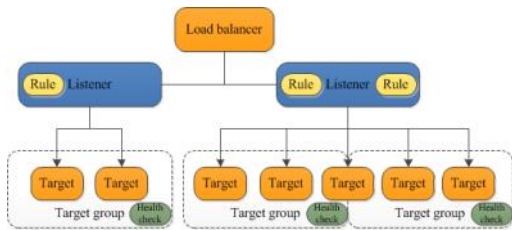
Application Load Balancer + ECS

Amazon Elastic Container Service (Amazon ECS) is a highly scalable, fast, container management service that makes it easy to run, stop, and manage Docker containers on a cluster. You can host your cluster on a serverless infrastructure that is managed by Amazon ECS by launching your services or tasks using the Fargate launch type. For more control over your infrastructure, you can host your tasks on a cluster of Amazon Elastic Compute Cloud (Amazon EC2) instances that you manage by using the EC2 launch type.



via - <https://aws.amazon.com/ecs/>

An Application load balancer distributes incoming application traffic across multiple targets, such as EC2 instances, in multiple Availability Zones. A listener checks for connection requests from clients, using the protocol and port that you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets. Each rule consists of a priority, one or more actions, and one or more conditions.



via - <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>

When you deploy your services using Amazon Elastic Container Service (Amazon ECS), you can use dynamic port mapping to support multiple tasks from a single service on the same container instance. Amazon ECS manages updates to your services by automatically registering and deregistering containers with your target group using the instance ID and port for each container.

How do I set up dynamic port mapping for Amazon ECS?

Last updated: 2020-05-12

I want to set up dynamic port mapping for my container instance in Amazon Elastic Container Service (Amazon ECS).

Short Description

The Classic Load Balancer doesn't allow you to run multiple copies of a task on the same instance. Instead, with the Classic Load Balancer, you must statically map port numbers on a container instance. However, an Application Load Balancer uses dynamic port mapping, so you can run multiple tasks from a single service on the same container instance.

via - <https://aws.amazon.com/premiumsupport/knowledge-center/dynamic-port-mapping-ecs>

Incorrect options:

Classic Load Balancer + Beanstalk - The Classic Load Balancer doesn't allow you to run multiple copies of a task on the same instance. Instead, with the Classic Load Balancer, you must statically map port numbers on a container instance. So this option is ruled out.

Application Load Balancer + Beanstalk - You can create docker environments that support multiple containers per Amazon EC2 instance with a multi-container Docker platform for Elastic Beanstalk. However, ECS gives you finer control.

Classic Load Balancer + ECS - The Classic Load Balancer doesn't allow you to run multiple copies of a task in the same instance. Instead, with the Classic Load Balancer, you must statically map port numbers on a container instance. So this option is ruled out.

Question 14

You have a workflow process that pulls code from AWS CodeCommit and deploys to EC2 instances associated with tag group ProdBuilders. You would like to configure the instances to archive no more than two application revisions to conserve disk space.

Which of the following will allow you to implement this?

- AWS CloudWatch Log Agent
- Have a load balancer in front of your instances
- Integrate with AWS CodePipeline
- CodeDeploy Agent

Overall explanation

Correct option:

"CodeDeploy Agent"

The CodeDeploy agent is a software package that, when installed and configured on an instance, makes it possible for that instance to be used in CodeDeploy deployments. The CodeDeploy agent archives revisions and log files on instances. The CodeDeploy agent cleans up these artifacts to conserve disk space. You can use the `:max_revisions:` option in the agent configuration file to specify the number of application revisions to the archive by entering any positive integer. CodeDeploy also archives the log files of the last successful deployment. That log file is always retained, even if the number of failed deployments exceeds the number of retained revisions. If no value is specified, CodeDeploy retains the five most recent revisions in addition to the currently deployed revision.

More info here:

Application Revision and Log File Cleanup

The CodeDeploy agent archives revisions and log files on instances. The CodeDeploy agent cleans up these artifacts to conserve disk space.

Application revision deployment logs: You can use the `:max_revisions:` option in the agent configuration file to specify the number of application revisions to archive by entering any positive integer. CodeDeploy also archives the log files for these revisions. All others are deleted, with the exception of the log file of the last successful deployment. That log file is always retained, even if the number of failed deployments exceeds the number of retained revisions. If no value is specified, CodeDeploy retains the five most recent revisions in addition to the currently deployed revision.

CodeDeploy logs: For Amazon Linux, Ubuntu Server, and RHEL instances, the CodeDeploy agent rotates the log files under the `/var/log/aws/codedeploy-agent` folder. The log file is rotated at 00:00:00 (instance time) daily. Log files are deleted after seven days. The naming pattern for rotated log files is `codedeploy-agent-YYYYMMDD_Log`.

via - <https://docs.aws.amazon.com/codedeploy/latest/userguide/codedeploy-agent.html>

Incorrect options:

AWS CloudWatch Log Agent - The CloudWatch Logs agent provides an automated way to send log data to CloudWatch Logs from Amazon EC2 instances. This is an incorrect choice for the current use case.

Integrate with AWS CodePipeline - AWS CodePipeline is a fully managed continuous delivery service that helps you automate your release pipelines for fast and reliable application and infrastructure updates. CodeCommit and CodePipeline are already integrated services. CodePipeline cannot help in version control and management of archives on an EC2 instance.

Have a load balancer in front of your instances - Load Balancer helps balance incoming traffic across different EC2 instances. It is an incorrect choice for the current use case.

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/codedeploy-agent.html>

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/AgentReference.html>

Question 15

A development team has deployed a REST API in Amazon API Gateway to two different stages - a test stage and a prod stage. The test stage is used as a test build and the prod stage as a stable build. After the updates have passed the test, the team wishes to promote the test stage to the prod stage.

Which of the following represents the optimal solution for this use-case?

- Deploy the API without choosing a stage. This way, the working deployment will be updated in all stages
- Update stage variable value from the stage name of test to that of prod
- API performance is optimized in a different way for prod environments. Hence, promoting test to prod is not correct. The promotion should be done by redeploying the API to the prod stage
- Delete the existing prod stage. Create a new stage with the same name (prod) and deploy the tested version on this stage

Overall explanation

Correct option:

Update stage variable value from the stage name of test to that of prod

After creating your API, you must deploy it to make it callable by your users. To deploy an API, you create an API deployment and associate it with a stage. A stage is a logical reference to a lifecycle state of your API (for example, dev, prod, beta, v2). API stages are identified by the API ID and stage name. They're included in the URL that you use to invoke the API. Each stage is a named reference to a deployment of the API and is made available for client applications to call.

Stages enable robust version control of your API. In our current use-case, after the updates pass the test, you can promote the test stage to the prod stage. The promotion can be done by redeploying the API to the prod stage or updating a stage variable value from the stage name of test to that of prod.

Incorrect options:

Deploy the API without choosing a stage. This way, the working deployment will be updated in all stages - An API can only be deployed to a stage. Hence, it is not possible to deploy an API without choosing a stage.

Delete the existing prod stage. Create a new stage with the same name (prod) and deploy the tested version on this stage* - This is possible, but not an optimal way of deploying a change. Also, as prod refers to real production system, this option will result in downtime.

API performance is optimized in a different way for prod environments. Hence, promoting test to prod is not correct. The promotion should be done by redeploying the API to the prod stage - For each stage, you can optimize API performance by adjusting the default account-level request throttling limits and enabling API caching. And these settings can be changed/updated at any time.

Question 16

A developer is designing an AWS CloudFormation template for deploying Amazon EC2 instances in numerous AWS accounts. The developer needs to select EC2 instances from a list of pre-approved instance types.

What measures could the developer take to integrate the list of authorized instance types into the CloudFormation template?

- Configure a mapping having a list of EC2 instance types as parameters in the CloudFormation template
- Configure a parameter with the list of EC2 instance types as AllowedValues in the CloudFormation template
- Configure separate parameters for each EC2 instance type in the CloudFormation template
- Configure a pseudo parameter with the list of EC2 instance types as AllowedValues in the CloudFormation template

Overall explanation

Correct option:

Configure a parameter with the list of EC2 instance types as AllowedValues in the CloudFormation template

You can use the Parameters section to customize your templates. Parameters enable you to input custom values to your template each time you create or update a stack.

Defining a parameter in a template

The following example declares a parameter named `InstanceTypeParameter`. This parameter lets you specify the Amazon EC2 instance type for the stack to use when you create or update the stack.

Note that `InstanceTypeParameter` has a default value of `t2.micro`. This is the value that AWS CloudFormation uses to provision the stack unless another value is provided.

JSON

```
"Parameters" : {
  "InstanceTypeParameter" : {
    "Type" : "String",
    "Default" : "t2.micro",
    "AllowedValues" : ["t2.micro", "m1.small", "m1.large"],
    "Description" : "Enter t2.micro, m1.small, or m1.large. Default is t2.micro."
  }
}
```

YAML

```
Parameters:
  InstanceTypeParameter:
    Type: String
    Default: t2.micro
    AllowedValues:
      - t2.micro
      - m1.small
      - m1.large
    Description: Enter t2.micro, m1.small, or m1.large. Default is t2.micro.
```

via - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/parameters-section-structure.html>

AllowedValues refers to an array containing the list of values allowed for the parameter. When applied to a parameter of type String, the parameter value must be one of the allowed values. When applied to a parameter of type CommaDelimitedList, each value in the list must be one of the specified allowed values.

Incorrect options:

Configure separate parameters for each EC2 instance type in the CloudFormation template - Creating separate parameters for each instance type is semantically incorrect as the underlying value will point to the same resource but have multiple inputs.

Configure a mapping having a list of EC2 instance types as parameters in the CloudFormation template - The Mappings section matches a key to a corresponding set of named values. For example, if you want to set values based on a region, you can create a mapping that uses the region name as a key and contains the values you want to specify for each specific region. You use the `Fn::FindInMap` intrinsic function to retrieve values in a map. A mapping is not a list, rather, it consists of key value pairs. You can't include parameters, pseudo parameters, or intrinsic functions in the Mappings section. So, this option is incorrect.

Configure a pseudo parameter with the list of EC2 instance types as AllowedValues in the CloudFormation template - Pseudo parameters are parameters that are predefined by AWS CloudFormation. You don't declare them in your template. So, this option is incorrect.

Question 17

A company follows collaborative development practices. The engineering manager wants to isolate the development effort by setting up simulations of API components owned by various development teams.

Which API integration type is best suited for this requirement?

- HTTP
- MOCK
- AWS_PROXY
- HTTP_PROXY

Overall explanation

Correct option:

MOCK

This type of integration lets API Gateway return a response without sending the request further to the backend. This is useful for API testing because it can be used to test the integration setup without incurring charges for using the backend and to enable collaborative development of an API.

In collaborative development, a team can isolate their development effort by setting up simulations of API components owned by other teams by using the MOCK integrations. It is also used to return CORS-related headers to ensure that the API method permits CORS access. In fact, the API Gateway console integrates the OPTIONS method to support CORS with a mock integration.

Incorrect options:

AWS_PROXY - This type of integration lets an API method be integrated with the Lambda function invocation action with a flexible, versatile, and streamlined integration setup. This integration relies on direct interactions between the client and the integrated Lambda function.

HTTP_PROXY - The HTTP proxy integration allows a client to access the backend HTTP endpoints with a streamlined integration setup on single API method. You do not set the integration request or the integration response. API Gateway passes the incoming request from the client to the HTTP endpoint and passes the outgoing response from the HTTP endpoint to the client.

HTTP - This type of integration lets an API expose HTTP endpoints in the backend. With the HTTP integration, you must configure both the integration request and integration response. You must set up necessary data mappings from the method request to the integration request, and from the integration response to the method response.

Question 18

An IT company is using AWS CloudFormation to manage its IT infrastructure. It has created a template to provision a stack with a VPC and a subnet. The output value of this subnet has to be used in another stack.

As a Developer Associate, which of the following options would you suggest to provide this information to another stack?

- Use 'Export' field in the Output section of the stack's template
- Use `Fn::Transform`
- Use 'Expose' field in the Output section of the stack's template
- Use `Fn::ImportValue`

Overall explanation

Correct option:

Use 'Export' field in the Output section of the stack's template

To share information between stacks, export a stack's output values. Other stacks that are in the same AWS account and region can import the exported values.

To export a stack's output value, use the Export field in the Output section of the stack's template. To import those values, use the `Fn::ImportValue` function in the template for the other stacks.

Incorrect options:

Use 'Expose' field in the Output section of the stack's template - 'Expose' is a made-up option, and only given as a distractor.

Use `Fn::ImportValue` - To import the values exported by another stack, we use the `Fn::ImportValue` function in the template for the other stacks. This function is not useful for the current scenario.

Use `Fn::Transform` - The intrinsic function `Fn::Transform` specifies a macro to perform custom processing on part of a stack template. Macros enable you to perform custom processing on templates, from simple actions like find-and-replace operations to extensive transformations of entire templates. This function is not useful for the current scenario.

Question 19

A development team is considering Amazon ElastiCache for Redis as its in-memory caching solution for its relational database.

Which of the following options are correct while configuring ElastiCache? (Select two)

- If you have no replicas and a node fails, you experience no loss of data when using Redis with cluster mode enabled
- You can scale write capacity for Redis by adding replica nodes
- While using Redis with cluster mode enabled, asynchronous replication mechanisms are used to keep the read replicas synchronized with the primary. If cluster mode is disabled, the replication mechanism is done synchronously
- While using Redis with cluster mode enabled, you cannot manually promote any of the replica nodes to primary
- All the nodes in a Redis cluster must reside in the same region

Overall explanation

Correct options:

All the nodes in a Redis cluster must reside in the same region

All the nodes in a Redis cluster (cluster mode enabled or cluster mode disabled) must reside in the same region.

While using Redis with cluster mode enabled, you cannot manually promote any of the replica nodes to primary

While using Redis with cluster mode enabled, there are some limitations:

4. You cannot manually promote any of the replica nodes to primary.
5. Multi-AZ is required.
6. You can only change the structure of a cluster, the node type, and the number of nodes by restoring from a backup.

Incorrect options:

While using Redis with cluster mode enabled, asynchronous replication mechanisms are used to keep the read replicas synchronized with the primary. If cluster mode is disabled, the replication mechanism is done synchronously - When you add a read replica to a cluster, all of the data from the primary is copied to the new node. From that point on, whenever data is written to the primary, the changes are asynchronously propagated to all the read replicas, for both the Redis offerings (cluster mode enabled or cluster mode disabled).

If you have no replicas and a node fails, you experience no loss of data when using Redis with cluster mode enabled - If you have no replicas and a node fails, you experience loss of all data in that node's shard, when using Redis with cluster mode enabled. If you have no replicas and the node fails, you experience total data loss in Redis with cluster mode disabled.

You can scale write capacity for Redis by adding replica nodes - This increases only the read capacity of the Redis cluster, write capacity is not enhanced by read replicas.

Question 20

A developer wants a seamless ability to return to older versions of a Lambda function that is being deployed.

Which of the following solutions offers the LEAST operational overhead?

Use Lambda function layers that can point to the different versions\

Use a Lambda function alias that can point to the different versions

Use CodeDeploy to configure blue/green deployments for the different Lambda function versions

Use a Route 53 weighted policy that can point to the different Lambda function versions

Overall explanation

Correct option:

Use a Lambda function alias that can point to the different versions

You can use versions to manage the deployment of your functions. For example, you can publish a new version of a function for beta testing without affecting users of the stable production version.

Lambda creates a new version of your function each time that you publish the function. The new version is a copy of the unpublished version of the function.

By publishing a version of your function, you can store your code and configuration as a separate resource that cannot be changed.

A Lambda alias is like a pointer to a specific function version. Users can access the function version using the alias Amazon Resource Name (ARN). Each alias has a unique ARN. An alias can point only to a function version, not to another alias. You can update an alias to point to the different versions of the Lambda function.

Incorrect options:

Use a Route 53 weighted policy that can point to the different Lambda function versions - This option is a distractor, as Route 53 cannot be used for the given use case. Route 53 weighted policy lets you associate multiple resources with a single domain name (example.com) or subdomain name (acme.example.com) and choose how much traffic is routed to each resource. This can be useful for a variety of purposes, including load balancing and testing new versions of software.

Use CodeDeploy to configure blue/green deployments for the different Lambda function versions - A deployment to the AWS Lambda compute platform is always a blue/green deployment.

You do not specify a deployment type option. When you deploy to an AWS Lambda compute platform, the deployment configuration specifies the way traffic is shifted to the new Lambda function versions in your application. You can shift traffic using a canary, linear, or all-at-once deployment configuration. Once deployed, you cannot go back to the previous versions of your Lambda function. So this option is incorrect.

Use Lambda function layers that can point to the different versions - Lambda layers provide a convenient way to package libraries and other dependencies that you can use with your Lambda functions. Using layers reduces the size of uploaded deployment archives and makes it faster to deploy your code. You cannot use the Lambda function layers to point to the different versions of the Lambda function.

Question 21

The development team at a company is looking at building an AWS CloudFormation template that self-populates the AWS Region variable while deploying the CloudFormation template.

What is the MOST operationally efficient way to determine the Region in which the template is being deployed?

- Create a CloudFormation parameter for Region and let the desired value be populated at the time of deployment
- Use the AWS::Region pseudo parameter
- Set up a mapping containing the key and the named values for all AWS Regions and then have the CloudFormation template auto-select the desired value
- Create an AWS Lambda-backed custom resource for Region and let the desired value be populated at the time of deployment by the Lambda

Overall explanation

Correct option:

Use the AWS::Region pseudo parameter

Pseudo parameters are parameters that are predefined by AWS CloudFormation. You don't declare them in your template. Use them the same way as you would a parameter, as the argument for the Ref function.

You can access pseudo parameters in a CloudFormation template like so:

Outputs:

MyStacksRegion:

Value: !Ref "AWS::Region"

The AWS::Region pseudo parameter returns a string representing the Region in which the encompassing resource is being created, such as us-west-2.

Incorrect options:

Create a CloudFormation parameter for Region and let the desired value be populated at the time of deployment - Although it is certainly possible to use a CloudFormation parameter to populate the desired value of the Region at the time of deployment, however, this is not operationally efficient, as you can directly use the AWS::Region pseudo parameter for this.

Set up a mapping containing the key and the named values for all AWS Regions and then have the CloudFormation template auto-select the desired value - The Mappings section matches a key to a corresponding set of named values. For example, if you want to set values based on a Region, you can create a mapping that uses the Region name as a key and contains the values you want to specify for each specific Region. You use the Fn::FindInMap intrinsic function to retrieve values in a map. This option is incorrect as the CloudFormation template cannot auto-select the desired value of the Region from a mapping.

Create an AWS Lambda-backed custom resource for Region and let the desired value be populated at the time of deployment by the Lambda - Custom resources enable you to write custom provisioning logic in templates that AWS CloudFormation runs anytime you create, update (if you changed the custom resource), or delete stacks. When you associate a Lambda function with a custom resource, the function is invoked whenever the custom resource is created, updated, or deleted. This option is a distractor, as Region is not a custom resource that needs to be provisioned.

Question 22

You have a Java-based application running on EC2 instances loaded with AWS CodeDeploy agents. You are considering different options for deployment, one is the flexibility that allows for incremental deployment of your new application versions and replaces existing versions in the EC2 instances. The other option is a strategy in which an Auto Scaling group is used to perform a deployment.

Which of the following options will allow you to deploy in this manner? (Select two)

- Warm Standby Deployment
- Pilot Light Deployment
- In-place Deployment
- Cattle Deployment
- Blue/green Deployment

Overall explanation

Correct options:

In-place Deployment

The application on each instance in the deployment group is stopped, the latest application revision is installed, and the new version of the application is started and validated. You can use a load balancer so that each instance is deregistered during its deployment and then restored to service after the deployment is complete.

Blue/green Deployment

With a blue/green deployment, you provision a new set of instances on which CodeDeploy installs the latest version of your application. CodeDeploy then re-routes load balancer traffic from an existing set of instances running the previous version of your application to the new set of instances running the latest version. After traffic is re-routed to the new instances, the existing instances can be terminated.

CodeDeploy Deployment Types:

Working with Deployments in CodeDeploy

en · 10 min · 100

In **CodeDeploy**, a deployment is the process, and the components involved in the process, of installing content on one or more instances. This content can consist of code, web site configuration files, application packages, scripts, and so on. CodeDeploy deploys content that is stored in a source repository, according to the configuration rules you specify.

If you use the EC2/On-Premises compute platform, then two deployments to the same set of instances can run concurrently.

CodeDeploy provides two deployment type options, in-place deployments and Blue/Green deployments.

- In-place deployment:** The application on each instance in the deployment group is replaced. The latest application version is installed, and the new version of the application is started and validated. You can use a test instance to first test the application is deployed during the deployment and then rolled back to previous after the deployment is complete. Only deployments that use the EC2/On-Premises compute platform can use in-place deployments. For more information about in-place deployments, see *Overview of an In-Place Deployment*.
- Blue/Green deployment:** The behavior of your deployment depends on which compute platform you use.
 - Blue/Green on an EC2/On-Premises compute platform:** The instances in a deployment group (the original environment) are replaced by a different set of instances (the replacement environment) using three steps:
 - Instances are provisioned for the replacement environment.
 - The latest application version is installed on the replacement instances.
 - An optional test step occurs for validation such as application testing and system verification.
 - Instances in the replacement environment are registered with an Elastic Load Balancing load balancer, causing traffic to be rerouted to them. Instances in the original environment are deregistered and can be terminated or kept running for other uses.

Note

If you use an EC2/On-Premises compute platform, be aware that Blue/Green deployments work with Amazon EC2 instances only.

- Blue/Green on an AWS Lambda compute platform:** Traffic is shifted from your current serverless environment to one with your updated Lambda function version. You can specify Lambda functions that perform validation tests and choose the way in which the traffic shifting occurs. All AWS Lambda compute platform deployments are Blue/Green deployments. For this reason, you do not need to specify a deployment type.
- Blue/Green on an Amazon ECS compute platform:** Traffic is shifted from the task set with the original version of an application to an Amazon ECS service to a replacement task set in the same service. You can set the traffic shifting to occur in stages through the deployment configuration. The protocol and port of a specified listener feature is used to reroute production traffic. During a deployment, a test listener can be used to serve traffic to the replacement task set while validation tests are run.
- Blue/Green deployments through AWS CloudFormation: Traffic is shifted from your current resources to your updated resources as part of an AWS CloudFormation stack update. Currently, only ECS Blue/Green deployments are supported.

via - <https://docs.aws.amazon.com/codedeploy/latest/userguide/deployments.html>

Incorrect options:

Cattle Deployment - This is a good option if you have cattle in a farm

Warm Standby Deployment - This is not a valid CodeDeploy deployment option. The term "Warm Standby" is used to describe a Disaster Recovery scenario in which a scaled-down version of a fully functional environment is always running in the cloud.

Pilot Light Deployment - This is not a valid CodeDeploy deployment option. "Pilot Light" is a Disaster Recovery approach where you simply replicate part of your IT structure for a limited set of core services so that the AWS cloud environment seamlessly takes over in the event of a disaster.

Question 23

You have been hired at a company that needs an experienced developer to help with a continuous integration/continuous delivery (CI/CD) workflow on AWS. You configure the company's workflow to run an AWS CodePipeline pipeline whenever the application's source code changes in a repository hosted in AWS Code Commit and compiles source code with AWS Code Build. You are configuring ProjectArtifacts in your build stage.

Which of the following should you do?

- Give AWS CodeBuild permissions to upload the build output to your Amazon S3 bucket
- Configure AWS CodeBuild to store output artifacts on EC2 servers
- Contact AWS Support to allow AWS CodePipeline to manage build outputs
- Give AWS CodeCommit permissions to upload the build output to your Amazon S3 bucket

Overall explanation

Correct option:

Give AWS CodeBuild permissions to upload the build output to your Amazon S3 bucket

If you choose ProjectArtifacts and your value type is S3 then the build project stores build output in Amazon Simple Storage Service (Amazon S3). For that, you will need to give AWS CodeBuild permissions to upload.

Incorrect options:

Configure AWS CodeBuild to store output artifacts on EC2 servers - EC2 servers are not a valid output location, so this option is ruled out.

Give AWS CodeCommit permissions to upload the build output to your Amazon S3 bucket - AWS CodeCommit is the repository that holds source code and has no control over compiling the source code, so this option is incorrect.

Contact AWS Support to allow AWS CodePipeline to manage build outputs - You can set AWS CodePipeline to manage its build output locations instead of AWS CodeBuild. There is no need to contact AWS Support.

Question 24

An e-commerce company has implemented AWS CodeDeploy as part of its AWS cloud CI/CD strategy. The company has configured automatic rollbacks while deploying a new version of its flagship application to Amazon EC2.

What occurs if the deployment of the new version fails?

- A new deployment of the last known working version of the application is deployed with a new deployment ID
- CodeDeploy switches the Route 53 alias records back to the known good green deployment and terminates the failed blue deployment
- AWS CodePipeline promotes the most recent working deployment with a SUCCEEDED status to production
- The last known working deployment is automatically restored using the snapshot stored in Amazon S3

Overall explanation

Correct option:

A new deployment of the last known working version of the application is deployed with a new deployment ID

AWS CodeDeploy is a service that automates code deployments to any instance, including Amazon EC2 instances and instances running on-premises. AWS CodeDeploy makes it easier for you to rapidly release new features, helps you avoid downtime during deployment, and handles the complexity of updating your applications.

CodeDeploy rolls back deployments by redeploying a previously deployed revision of an application as a new deployment. These rolled-back deployments are technically new deployments, with new deployment IDs, rather than restored versions of a previous deployment.

To roll back an application to a previous revision, you just need to deploy that revision. AWS CodeDeploy keeps track of the files that were copied for the current revision and removes them before starting a new deployment, so there is no difference between redeploy and rollback. However, you need to make sure that the previous revisions are available for rollback.

Incorrect options:

The last known working deployment is automatically restored using the snapshot stored in Amazon S3 - CodeDeploy deployment does not have a snapshot stored on S3, so this option is incorrect.

AWS CodePipeline promotes the most recent working deployment with a SUCCEEDED status to production - The use-case does not talk about using CodePipeline, so this option just acts as a distractor.

CodeDeploy switches the Route 53 alias records back to the known good green deployment and terminates the failed blue deployment - The use-case does not talk about the blue/green deployment, so this option has just been added as a distractor.

Question 25

Your company is in the process of building a DevOps culture and is moving all of its on-premise resources to the cloud using serverless architectures and automated deployments. You have created a CloudFormation template in YAML that uses an AWS Lambda function to pull HTML files from GitHub and place them into an Amazon Simple Storage Service (S3) bucket that you specify.

Which of the following AWS CLI commands can you use to upload AWS Lambda functions and AWS CloudFormation templates to AWS?

- cloudformation package and cloudformation deploy
- cloudformation zip and cloudformation upload
- cloudformation package and cloudformation upload
- cloudformation zip and cloudformation deploy

Overall explanation

Correct option:

cloudformation package and cloudformation deploy

AWS CloudFormation gives developers and businesses an easy way to create a collection of related AWS and third-party resources and provision them in an orderly and predictable fashion.

How CloudFormation Works:



via - <https://aws.amazon.com/cloudformation/>

The cloudformation package command packages the local artifacts (local paths) that your AWS CloudFormation template references. The command will upload local artifacts, such as your source code for your AWS Lambda function.

The cloudformation deploy command deploys the specified AWS CloudFormation template by creating and then executing a changeset.

Incorrect options:

cloudformation package and cloudformation upload - The cloudformation upload command does not exist.

cloudformation zip and cloudformation upload - Both commands do not exist, this is a made-up option.

cloudformation zip and cloudformation deploy - The cloudformation zip command does not exist, this is a made-up option.

Question 26

Your organization has set up a full CI/CD pipeline leveraging CodePipeline and the deployment is done on Elastic Beanstalk. This pipeline has worked for over a year now but you are approaching the limits of Elastic Beanstalk in terms of how many versions can be stored in the service.

How can you remove older versions that are not used by Elastic Beanstalk so that new versions can be created for your applications?

- Define a Lambda function
- Setup an .ebextensions file
- Use Worker Environments
- Use a Lifecycle Policy

Overall explanation

Correct option:

Use a Lifecycle Policy

Each time you upload a new version of your application with the Elastic Beanstalk console or the EB CLI, Elastic Beanstalk creates an application version. If you don't delete versions that you no longer use, you will eventually reach the application version limit and be unable to create new versions of that application.

You can avoid hitting the limit by applying an application version lifecycle policy to your applications. A lifecycle policy tells Elastic Beanstalk to delete old application versions or to delete application versions when the total number of versions for an application exceeds a specified number.

Elastic Beanstalk applies an application's lifecycle policy each time you create a new application version and deletes up to 100 versions each time the lifecycle policy is applied. Elastic Beanstalk deletes old versions after creating the new version and does not count the new version towards the maximum number of versions defined in the policy.

Configuring application version lifecycle settings

Each time you upload a new version of your application with the Elastic Beanstalk console or the EB CLI, Elastic Beanstalk creates an application version. If you don't delete versions that you no longer use, you will eventually reach the application version quota and be unable to create new versions of that application.

You can avoid hitting the quota by applying an application version lifecycle policy to your applications. A lifecycle policy tells Elastic Beanstalk to delete application versions that are old, or to delete application versions when the total number of versions for an application exceeds a specified number.

Elastic Beanstalk applies an application's lifecycle policy each time you create a new application version, and deletes up to 100 versions each time the lifecycle policy is applied. Elastic Beanstalk deletes old versions after creating the new version, and does not count the new version towards the maximum number of versions defined in the policy.

Elastic Beanstalk does not delete application versions that are currently being used by an environment, or application versions deployed to environments that were terminated less than ten weeks before the policy was triggered.

The application-version quota applies across all applications in a region. If you have several applications, configure each application with a lifecycle policy appropriate to avoid reaching the quota. For example, if you have 10 applications in a region and the quota is 1,000 application versions, consider setting a lifecycle policy with a quota of 90 application versions for all applications, or set other values in each application as long as the total is less than 1,000 application versions. Elastic Beanstalk only applies the policy if the application version creation succeeds, so if you have already reached the quota, you must delete some versions manually prior to creating a new version.

By default, Elastic Beanstalk leaves the application version's source bundle in Amazon S3 to prevent loss of data. You can delete the source bundle to save space.

You can set the lifecycle settings through the Elastic Beanstalk CLI and APIs. See `eb applications`, `CreateApplication` (using the `ResourceLifecycleConfiguration` parameter), and `UpdateApplicationResourceLifecycle` for details.

via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/applications-lifecycle.html>

Incorrect options: **Setup an .ebextensions files** - You can add AWS Elastic Beanstalk configuration files (.ebextensions) to your web application's source code to configure your environment and customize the AWS resources that it contains. This does not help with managing versions.

Define a Lambda function - This could work but would require a lot of manual scripting, to achieve the same desired effect as the Lifecycle Policy EB feature.

Use Worker Environments - This won't help. If your application performs operations or workflows that take a long time to complete, you can offload those tasks to a dedicated worker environment. Decoupling your web application front end from a process that performs blocking operations is a common way to ensure that your application stays responsive under load.

Question 27

An e-commerce company has deployed its application on AWS Elastic Beanstalk. The Auto Scaling group associated with the Beanstalk environment has three Amazon EC2 instances. When the number of instances falls below two, it severely impacts the performance of the web application. The company currently uses the default all-at-once deployment policy and is looking for an effective strategy for future deployments.

Which of the following represents the most cost-effective deployment strategy for the company?

- Configure an Elastic Load Balancer to front the Auto Scaling Group and choose two different Availability Zones (AZs) for deployment
- Opt for rolling with additional batch deployment strategy. Set the batch size parameter to 1
- Opt for traffic-splitting deployment strategy with traffic split parameter set to 50% of the total traffic
- Opt for rolling deployment strategy. Set the batch size to 2

Overall explanation

Correct option:

Opt for rolling with additional batch deployment strategy. Set the batch size parameter to 1

With rolling deployments, Elastic Beanstalk splits the environment's Amazon EC2 instances into batches and deploys the new version of the application to one batch at a time. It leaves the rest of the instances in the environment running the old version of the application. During a rolling deployment, some instances serve requests with the old version of the application, while instances in completed batches serve other requests with the new version.

To maintain full capacity during deployments, you can configure your environment to launch a new batch of instances before taking any instances out of service. This option is known as a rolling deployment with an additional batch. When the deployment completes, Elastic Beanstalk terminates the additional batch of instances.

For the rolling and rolling with additional batch deployment policies, you can configure Batch size - The size of the set of instances to deploy in each batch. Choose Percentage to configure a percentage of the total number of EC2 instances in the Auto Scaling group (up to 100 percent), or choose Fixed to configure a fixed number of instances (up to the maximum instance count in your environment's Auto Scaling configuration).

Example configuration for rolling With additional batch:

To deploy to five instances in each batch, regardless of the number of instances running, and to bring up an extra batch of five instances running the new version before pulling any instances out of service, specify the following options and values.

Example .ebextensions/rolling-additionalbatch.config

```
option_settings:
  aws:elasticbeanstalk:command:
    DeploymentPolicy: RollingWithAdditionalBatch
    BatchSizeType: Fixed
    BatchSize: 5
```

via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html#environments-cfg-rollingdeployments-method>

Incorrect options:

Opt for a traffic-splitting deployment strategy with a traffic split parameter set to 50% of the total traffic - Traffic-splitting deployments let you perform canary testing as part of your application deployment. In a traffic-splitting deployment, Elastic Beanstalk launches a full set of new instances just like during an immutable deployment. It then forwards a specified percentage of incoming client traffic to the new application version for a specified evaluation period. This turns out to be a costly solution for the given use case.

Configure an Elastic Load Balancer to front the Auto Scaling Group and choose two different Availability Zones (AZs) for deployment - Adding AZs makes the entire configuration resilient for failures post-deployment. However, it does not represent a valid deployment strategy for the given use case. This option has been added as a distractor.

Opt for a rolling deployment strategy. Set the batch size to 2 - As already discussed, with rolling deployments, Elastic Beanstalk splits the environment's Amazon EC2 instances into batches and deploys the new version of the application to one batch at a time. With a batch size set to 2, only 1 instance will be left for traffic during deployment, impacting the performance of the e-commerce application. Hence, this option is incorrect.

Question 28

You would like to have a one-stop dashboard for all the CI/CD needs of one of your projects. You don't need heavy control of the individual configuration of each component in your CI/CD, but need to be able to get a holistic view of your projects.

Which service do you recommend?

- CodeStar
- CodeDeploy
- CodeBuild
- CodePipeline

Overall explanation

Correct option:

CodeStar

AWS CodeStar enables you to quickly develop, build, and deploy applications on AWS. AWS CodeStar provides a unified user interface, enabling you to easily manage your software development activities in one place. With AWS CodeStar, you can set up your entire continuous delivery toolchain in minutes, allowing you to start releasing code faster. AWS CodeStar makes it easy for your whole team to work together securely, allowing you to easily manage access and add owners, contributors, and viewers to your projects. Each AWS CodeStar project comes with a project management dashboard, including an integrated issue tracking capability powered by Atlassian JIRA Software. With the AWS CodeStar project dashboard, you can easily track progress across your entire software development process, from your backlog of work items to teams' recent code deployments.

Incorrect options:

CodeBuild

CodeDeploy

CodePipeline

All these options are individual services encompassed by CodeStar when you deploy a project. They have to be used individually and don't provide a unified "project" view.

Question 29

Your company is shifting towards Elastic Container Service (ECS) to deploy applications. The process should be automated using the AWS CLI to create a service where at least ten instances of a task definition are kept running under the default cluster.

Which of the following commands should be executed?

- `aws ecs run-task --cluster default --task-definition ecs-demo`
- `aws ecr create-service --service-name ecs-simple-service --task-definition ecs-demo --desired-count 10`
- `aws ecs create-service --service-name ecs-simple-service --task-definition ecs-demo --desired-count 10`
- `docker-compose create ecs-simple-service`

Overall explanation

Correct option:

aws ecs create-service --service-name ecs-simple-service --task-definition ecs-demo --desired-count 10

To create a new service you would use this command which creates a service in your default region called ecs-simple-service. The service uses the ecs-demo task definition and it maintains 10 instantiations of that task.

Incorrect options:

aws ecr create-service --service-name ecs-simple-service --task-definition ecs-demo --desired-count 10 - This command is referencing a different service called Amazon Elastic Container Registry (ECR) which is a fully-managed Docker container registry

docker-compose create ecs-simple-service - This is a docker command to create containers for a service.

aws ecs run-task --cluster default --task-definition ecs-demo - This is a valid command but used for starting a new task using a specified task definition.

Question 30

You have created a test environment in Elastic Beanstalk and as part of that environment, you have created an RDS database.

How can you make sure the database can be explored after the environment is destroyed?

- Convert the Elastic Beanstalk environment to a worker environment
- Make a snapshot of the database before it gets deleted
- Change the Elastic Beanstalk environment variables
- Make a selective delete in Elastic Beanstalk

Overall explanation

Correct option:

Make a snapshot of the database before it gets deleted

Use an Elastic Beanstalk blue (environment A)/green (environment B) deployment to decouple an RDS DB instance from environment.

Create a new Elastic Beanstalk environment (environment B) with the necessary information to connect to the RDS DB instance.

Note: An RDS DB instance attached to an Elastic Beanstalk environment is ideal for development and testing environments. However, it's not ideal for production environments because the lifecycle of the database instance is tied to the lifecycle of your application environment. If you terminate the environment, then you lose your data because the RDS DB instance is deleted by the environment. For more information, see Using Elastic Beanstalk with Amazon RDS.

This is the only way to recover the database data before it gets deleted by Elastic Beanstalk.

Please review this excellent document that addresses this use-case :

<https://aws.amazon.com/premiumsupport/knowledge-center/decouple-rds-from-beanstalk/>

Incorrect options:

Make a selective delete in Elastic Beanstalk - This is not a feature in Elastic Beanstalk.

Change the Elastic Beanstalk environment variables - Environment variables won't help with the provisioned RDS database.

Convert the Elastic Beanstalk environment to a worker environment - You can't convert Elastic Beanstalk environments, you can only change their settings.

Question 31

Your team lead has finished creating a CodeBuild project in the management console and a build spec has been defined for the project. After the build is run, CodeBuild fails to pull a Docker image into the build environment.

What is the most likely cause?

- The Docker image is too big
- The Docker image is missing some tags
- Missing IAM permissions for the CodeBuild Service
- CodeBuild cannot work with custom Docker images

Overall explanation

Correct option:

Missing IAM permissions for the CodeBuild Service

By default, IAM users don't have permission to create or modify Amazon Elastic Container Registry (Amazon ECR) resources or perform tasks using the Amazon ECR API. A user who uses the AWS CodeBuild console must have a minimum set of permissions that allows the user to describe other AWS resources for the AWS account.

3. If any of the following is true, you must add permissions to your image repository in Amazon ECR so that AWS CodeBuild can pull its Docker image into the build environment.

- Your project uses CodeBuild credentials to pull Amazon ECR images. This is denoted by a value of CODEBUILD in the ImagePullCredentialsType attribute of your ProjectConfiguration.
- Your project uses a cross-account Amazon ECR image. In this case, your project must use its service role to pull Amazon ECR images. To enable this behavior, set the ImagePullCredentialsType attribute of your ProjectConfiguration to SERVICE_ROLE.

- a. Open the Amazon ECR console at <https://console.aws.amazon.com/ecr/>.
- b. In the list of repository names, choose the name of the repository you created or selected.
- c. From the repository page, choose Permissions. Choose RBAC, and then choose Add statement.
- d. For Statement name, enter an identifier (for example, CodeBuildAccess).
- e. For Effect, leave Allow selected. This indicates that you want to allow access to another AWS account.
- f. For Principal, do one of the following:

- If your project uses CodeBuild credentials to pull an Amazon ECR image in Service permissions, enter CODEBUILD.amazonaws.com.
- If your project uses a cross-account Amazon ECR image, for AWS account ID, enter ID of the AWS account that you want to give access.

- g. Specify the ARN within the list.
- h. For Action, enter the pull-only actions: `ecr:GetAuthorizationToken`, `ecr:BatchGetImage`, and `ecr:BatchCheckLayerAvailability`.
- i. Choose Save.

This policy is displayed in Permissions. The principal is what you entered for Principal in step f of this procedure.

- If your project uses CodeBuild credentials to pull an Amazon ECR image, "CODEBUILD.amazonaws.com" appears under Service principals.
- If your project uses a cross-account Amazon ECR image, the ID of the AWS account that you want to give access appears under AWS Account IDs.

The following sample policy uses both CodeBuild credentials and a cross-account Amazon ECR image.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ECRAccessPrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "CODEBUILD.amazonaws.com"
      },
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:BatchGetImage",
        "ecr:BatchCheckLayerAvailability"
      ]
    },
    {
      "Sid": "ECRAccessCrossAccount",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::AWS_ACCOUNT_ID:root"
      },
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:BatchGetImage",
        "ecr:BatchCheckLayerAvailability"
      ]
    }
  ]
}
```

via - <https://docs.aws.amazon.com/codebuild/latest/userguide/sample-ecr.html>

Incorrect options:

The Docker image is missing some tags - Tags are optional for naming purposes

CodeBuild cannot work with custom Docker images - Custom docker images are supported, so this option is incorrect.

The Docker image is too big - It is good to properly design the image but in this case, it does not affect the CodeBuild. You can also look at multi-stage builds, which are a new feature requiring

Docker 17.05 or higher on the daemon and client. Multistage builds are useful to anyone who has struggled to optimize Dockerfiles while keeping them easy to read and maintain.

Question 32

You are working for a technology startup building web and mobile applications. You would like to pull Docker images from the ECR repository called demo so you can start running local tests against the latest application version.

Which of the following commands must you run to pull existing Docker images from ECR? (Select two)

- `docker build -t 1234567890.dkr.ecr.eu-west-1.amazonaws.com/demo:latest`
- `aws docker push 1234567890.dkr.ecr.eu-west-1.amazonaws.com/demo:latest`
- `docker pull 1234567890.dkr.ecr.eu-west-1.amazonaws.com/demo:latest`
- `docker login -u $AWS_ACCESS_KEY_ID -p $AWS_SECRET_ACCESS_KEY`
- `$(aws ecr get-login --no-include-email)`

Overall explanation

Correct options:

`$(aws ecr get-login --no-include-email)`

`docker pull 1234567890.dkr.ecr.eu-west-1.amazonaws.com/demo:latest`

The get-login command retrieves a token that is valid for a specified registry for 12 hours, and then it prints a docker login command with that authorization token. You can execute the printed command to log in to your registry with Docker, or just run it automatically using the `$( )` command wrapper. After you have logged in to an Amazon ECR registry with this command, you can use the Docker CLI to push and pull images from that registry until the token expires. The docker pull command is used to pull an image from the ECR registry.

Incorrect options:

`docker login -u $AWS_ACCESS_KEY_ID -p $AWS_SECRET_ACCESS_KEY` - You cannot login to AWS ECR this way. `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` are only used by the CLI and not by docker.

`aws docker push 1234567890.dkr.ecr.eu-west-1.amazonaws.com/demo:latest` - docker push here is the wrong answer, you need to use docker pull.

`docker build -t 1234567890.dkr.ecr.eu-west-1.amazonaws.com/demo:latest` - This is a docker command that is used to build Docker images from a Dockerfile.

Question 33

A developer is creating a RESTful API service using an Amazon API Gateway with AWS Lambda integration. The service must support different API versions for testing purposes. As a Developer Associate, which of the following would you suggest as the best way to accomplish this?

- Use an X-Version header to identify which version is being called and pass that header to the Lambda function
- Deploy the API versions as unique stages with unique endpoints and use stage variables to provide the context to identify the API versions
- Use an API Gateway Lambda authorizer to route API clients to the correct API version
- Set up an API Gateway resource policy to identify the API versions and provide context to the Lambda function

Overall explanation

Correct option:

Deploy the API versions as unique stages with unique endpoints and use stage variables to provide the context to identify the API versions - A stage is a named reference to a deployment, which is a snapshot of the API. You use a stage to manage and optimize a particular deployment. For example, you can configure stage settings to enable caching, customize request throttling, configure logging, define stage variables, or attach a canary release for testing.

Stage variables are name-value pairs that you can define as configuration attributes associated with a deployment stage of a REST API. They act like environment variables and can be used in your API setup and mapping templates.

With deployment stages in API Gateway, you can manage multiple release stages for each API, such as alpha, beta, and production. Using stage variables you can configure an API deployment stage to interact with different backend endpoints.

For example, your API can pass a GET request as an HTTP proxy to the backend web host (for example, <http://example.com>). In this case, the backend web host is configured in a stage variable so that when developers call your production endpoint, API Gateway calls example.com. When you call your beta endpoint, API Gateway uses the value configured in the stage variable for the beta stage, and calls a different web host (for example, beta.example.com). Similarly, stage variables can be used to specify a different AWS Lambda function name for each stage in your API.

Incorrect options:

Use an X-Version header to identify which version is being called and pass that header to the Lambda function - This is an incorrect option and has been added as a distractor.

Use an API Gateway Lambda authorizer to route API clients to the correct API version - A Lambda authorizer is an API Gateway feature that uses a Lambda function to control access to your API. A Lambda authorizer is useful if you want to implement a custom authorization scheme that uses a bearer token authentication strategy such as OAuth or SAML, or that uses request parameters to determine the caller's identity.

Set up an API Gateway resource policy to identify the API versions and provide context to the Lambda function - Amazon API Gateway resource policies are JSON policy documents that you attach to an API to control whether a specified principal (typically an IAM user or role) can invoke the API. You can use API Gateway resource policies to allow your API to be securely invoked requestors. They are not meant for choosing the version of APIs.

Question 34

As a site reliability engineer, you are responsible for improving the company's deployment by scaling and automating applications. As new application versions are ready for production you ensure that the application gets deployed to different sets of EC2 instances at different times allowing for a smooth transition.

Using AWS CodeDeploy, which of the following options will allow you to do this?

- CodeDeploy Hooks
- CodeDeploy Deployment Groups
- CodeDeploy Agent
- Define multiple CodeDeploy Applications

Overall explanation

Correct option:

CodeDeploy Deployment Groups

You can specify one or more deployment groups for a CodeDeploy application. The deployment group contains settings and configurations used during the deployment. Most deployment group settings depend on the compute platform used by your application. Some settings, such as rollbacks, triggers, and alarms can be configured for deployment groups for any compute platform.

In an EC2/On-Premises deployment, a deployment group is a set of individual instances targeted for deployment. A deployment group contains individually tagged instances, Amazon EC2 instances in Amazon EC2 Auto Scaling groups, or both.

Incorrect options:

CodeDeploy Agent - The CodeDeploy agent is a software package that, when installed and configured on an instance, makes it possible for that instance to be used in CodeDeploy deployments. The agent connects the EC2 instances to the CodeDeploy service.

CodeDeploy Hooks - Hooks are found in the AppSec file used by AWS CodeDeploy to manage deployment. Hooks correspond to lifecycle events such as ApplicationStart, ApplicationStop, etc. to which you can assign a script.

Define multiple CodeDeploy Applications - This option has been added as a distractor. Instead, you want to use deployment groups to use the same deployment and maybe separate the times when a group of instances receives the software updates.

Question 35

The development team at an IT company uses CloudFormation to manage its AWS infrastructure. The team has created a network stack containing a VPC with subnets and a web application stack with EC2 instances and an RDS instance. The team wants to reference the VPC created in the network stack into its web application stack.

As a Developer Associate, which of the following solutions would you recommend for the given use-case?

- Create a cross-stack reference and use the Export output field to flag the value of VPC from the network stack. Then use `Fn::ImportValue` intrinsic function to import the value of VPC into the web application stack
- Create a cross-stack reference and use the Outputs output field to flag the value of VPC from the network stack. Then use `Ref` intrinsic function to reference the value of VPC into the web application stack
- Create a cross-stack reference and use the Outputs output field to flag the value of VPC from the network stack. Then use `Fn::ImportValue` intrinsic function to import the value of VPC into the web application stack
- Create a cross-stack reference and use the Export output field to flag the value of VPC from the network stack. Then use `Ref` intrinsic function to reference the value of VPC into the web application stack

Overall explanation

Correct option:

Create a cross-stack reference and use the Export output field to flag the value of VPC from the network stack. Then use `Fn::ImportValue` intrinsic function to import the value of VPC into the web application stack

AWS CloudFormation gives developers and businesses an easy way to create a collection of related AWS and third-party resources and provision them in an orderly and predictable fashion.

How CloudFormation Works:



via - <https://aws.amazon.com/cloudformation/>

You can create a cross-stack reference to export resources from one AWS CloudFormation stack to another. For example, you might have a network stack with a VPC and subnets and a separate public web application stack. To use the security group and subnet from the network stack, you can create a cross-stack reference that allows the web application stack to reference resource outputs from the network stack. With a cross-stack reference, owners of the web application stacks don't need to create or maintain networking rules or assets.

To create a cross-stack reference, use the Export output field to flag the value of a resource output for export. Then, use the Fn::ImportValue intrinsic function to import the value. You cannot use the Ref intrinsic function to import the value.

Walkthrough: Refer to resource outputs in another AWS CloudFormation stack

PDF | Kindle | RSS

To export resources from one AWS CloudFormation stack to another, create a cross-stack reference. Cross-stack references let you use a layered or service-oriented architecture. Instead of including all resources in a single stack, you create related AWS resources in separate stacks; then you can refer to required resource outputs from other stacks. By restricting cross-stack references to outputs, you control the parts of a stack that are referenced by other stacks.

For example, you might have a network stack with a VPC, a security group, and a subnet for public web applications, and a separate public web application stack. To ensure that the web applications use the security group and subnet from the network stack, you create a cross-stack reference that allows the web application stack to reference resource outputs from the network stack. With a cross-stack reference, owners of the web application stacks don't need to create or maintain networking rules or assets.

To create a cross-stack reference, use the Export output field to flag the value of a resource output for export. Then, use the Fn::ImportValue intrinsic function to import the value. For more information, see [Outputs and Fn::ImportValue](#).

via - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/walkthrough-crossstackref.html>

Incorrect options:

Create a cross-stack reference and use the Outputs output field to flag the value of VPC from the network stack. Then use Fn::ImportValue intrinsic function to import the value of VPC into the web application stack

Create a cross-stack reference and use the Outputs output field to flag the value of VPC from the network stack. Then use Ref intrinsic function to reference the value of VPC into the web application stack

Create a cross-stack reference and use the Export output field to flag the value of VPC from the network stack. Then use Ref intrinsic function to reference the value of VPC into the web application stack

These three options contradict the details provided in the explanation above, so these options are not correct.

Question 36

The development team at an e-commerce company wants to run a serverless data store service on two docker containers that share resources. Which of the following ECS configurations can be used to facilitate this use-case?

- Put the two containers into two separate task definitions using an EC2 Launch Type
- Put the two containers into two separate task definitions using a Fargate Launch Type
- Put the two containers into a single task definition using a Fargate Launch Type
- Put the two containers into a single task definition using an EC2 Launch Type

Overall explanation

Correct option:

Put the two containers into a single task definition using a Fargate Launch Type

Amazon Elastic Container Service (Amazon ECS) is a highly scalable, fast, container management service that makes it easy to run, stop, and manage Docker containers on a cluster. You can host your cluster on a serverless infrastructure that is managed by Amazon ECS by launching your services or tasks using the Fargate launch type. For more control over your infrastructure, you can host your tasks on a cluster of Amazon Elastic Compute Cloud (Amazon EC2) instances that you manage by using the EC2 launch type.



via - <https://aws.amazon.com/ecs/>

As the development team is looking for a serverless data store service, therefore the two containers should be launched into a single task definition using a Fargate Launch Type. Using a single task definition allows the two containers to share resources. Please see these use-cases for Fargate Launch type when you should put multiple containers into the same task definition:

Using the Fargate launch type

When architecting your application using the Fargate launch type for your tasks, the main question is when should you put multiple containers into the same task definition versus deploying containers separately in multiple task definitions.

You should put multiple containers in the same task definition if:

- Containers share a common lifecycle (that is, they should be launched and terminated together).
- Containers are required to be run on the same underlying host (that is, one container references the other on a localhost port).
- You want your containers to share resources.
- Your containers share data volumes.

Otherwise, you should define your containers in separate task definitions so that you can scale, provision, and deprovision them separately.

via - https://docs.aws.amazon.com/AmazonECS/latest/developerguide/application_architecture.html

For a deep-dive on understanding how Amazon ECS manages CPU and memory resources, please review this excellent blog- <https://aws.amazon.com/blogs/containers/how-amazon-ecs-manages-cpu-and-memory-resources/>

Incorrect options:

Put the two containers into two separate task definitions using a Fargate Launch Type - This option contradicts the details provided in the explanation above, so this option is ruled out.

Put the two containers into two separate task definitions using an EC2 Launch Type

Put the two containers into a single task definition using an EC2 Launch Type

As the development team is looking for a serverless data store service, therefore EC2 Launch Type is ruled out. So both these options are incorrect.

Question 37

A company wants to automate the creation of ECS clusters using CloudFormation. The process has worked for a while, but after creating task definitions and assigning roles, the development team discovers that the tasks for containers are not using the permissions assigned to them.

Which ECS config must be set in `/etc/ecs/ecs.config` to allow ECS tasks to use IAM roles?

- ECS_ENABLE_TASK_IAM_ROLE
- ECS_AVAILABLE_LOGGING_DRIVERS
- ECS_CLUSTER
- ECS_ENGINE_AUTH_DATA

Overall explanation

Correct option:

ECS_ENABLE_TASK_IAM_ROLE

This configuration item is used to enable IAM roles for tasks for containers with the bridge and default network modes.

Incorrect options:

ECS_ENGINE_AUTH_DATA - This refers to the authentication data within a Docker configuration file, so this is not the correct option.

ECS_AVAILABLE_LOGGING_DRIVERS - The Amazon ECS container agent running on a container instance must register the logging drivers available on that instance with this variable. This configuration item refers to the logging driver.

ECS_CLUSTER - This refers to the ECS cluster that the ECS agent should check into. This is passed to the container instance at launch through Amazon EC2 user data.

Question 38

Your e-commerce company needs to improve its software delivery process and is moving away from the waterfall methodology. You decided that every application should be built using the best CI/CD practices and every application should be packaged and deployed as a Docker container. The Docker images should be stored in ECR and pushed with AWS CodePipeline and AWS CodeBuild.

When you attempt to do this, the last step fails with an authorization issue. What is the most likely issue?

- The ECS instances are misconfigured and must contain additional data in `/etc/ecs/ecs.config`
- CodeBuild cannot talk to ECR because of security group issues
- The IAM permissions are wrong for the CodeBuild service
- The ECR repository is stale, you must delete and re-create it

Overall explanation

Correct option:

The IAM permissions are wrong for the CodeBuild service

You can push your Docker or Open Container Initiative (OCI) images to an Amazon ECR repository with the docker push command.

Amazon ECR users require permission to call `ecr:GetAuthorizationToken` before they can authenticate to a registry and push or pull any images from any Amazon ECR repository. Amazon ECR provides several managed policies to control user access at varying levels

Incorrect options:

The ECR repository is stale, you must delete and re-create it - You can delete a repository when you are done using it, stale is not a concept within ECR. This option has been added as a distractor.

CodeBuild cannot talk to ECR because of security group issues - A security group acts as a virtual firewall at the instance level and it is not related to pushing Docker images, so this option does not fit the given use-case.

The ECS instances are misconfigured and must contain additional data in `/etc/ecs/ecs.config` - The error Authorization is an indication that there is an access issue, therefore you should not look at your configuration first but rather permissions.

References:

<https://docs.aws.amazon.com/AmazonECR/latest/userguide/docker-push-ecr-image.html>

https://docs.aws.amazon.com/AmazonECR/latest/userguide/ecr_managed_policies.html

Domain

Deployment

From <<https://www.udemy.com/course/aws-certified-developer-associate-practice-tests-dva-c01/learn/quiz/5249670/result/1739521331?expanded=1739521331#overview>>